Instruction Selection & Scheduling via SMT

CS517
Raine Wheary, Christine Lin
GitHub project

We consider the problem of backend code generation in a compiler. By this point in the pipeline, we can understand the input program as having been reduced to a graph of basic operations in some intermediate representation (IR). The backend must further lower these operations into machine instructions.

For linear control flow (i.e. within a single basic block), there are three major tasks to be solved:

- **Instruction selection:** Modern CPU architectures often have complex macroinstructions encoding more than one basic operation. For example, the x86-64 instruction `mov rax, [rcx + 4*rdx + 28]` involves a left shift, two additions, and a load. Other examples include stack operations, multiple loads and stores (e.g. ARM's `ldp` and `stp`), and fused-multiply-add. A compiler will prefer to emit these instructions when applicable, but it can be a net loss if it means intermediate results are no longer available and need to be recomputed. In the last example, if the address value `rcx + 4*rdx + 28` was used in many places, a compiler should compute this effective address with `lea` and reuse it, rather than doing the calculation inside the `mov`.
- **Instruction scheduling:** Even when two machine instructions could are independent of each other, their order relative to each other can impact the performance of the generated code. We model a pipelined but not out-of-order processor, where a pipeline stall occurs if an instruction's inputs are not available.
- **Register allocation:** Programs in IR form use an unlimited number of virtual registers, but these need to be mapped to a limited number of physical registers on the machine.

Each of these tasks should be understood as a search problem. Classically, the backend is split into three phases, with instruction selection, instruction scheduling, and register allocation being performed in order [1]. However, this architecture has its limitations. When deciding whether to emit a higher-latency macroinstruction, there are situations where the optimal choice depends on register pressure or on the ability of the scheduler to avoid a pipeline stall.

Register allocation is well known to be NP-complete, and is typically solved with heuristic algorithms in practice. We will focus on an integrated algorithm for first two problems via reduction to SMT.

Problem statement

We will adopt the convention of coloring words and variables related to the IR (the input) blue and those related to the machine program (the output) red.

1) The IR program

We assume there is a fixed finite set of **IR opcodes** C and each opcode $\alpha \in C$ has an associated **arity**.

As input, we start with an **IR program**, which is a sequence P of **IR instructions**. Each IR instruction consists of an opcode and a tuple of arguments matching the opcode's arity.

$$P := \{P_1, ..., P_N\}$$
$$P_i := \alpha(t_1, ..., t_m), \qquad \alpha \in C, \quad m = \text{arity}(\alpha), \quad 1 \le t_1, ..., t_m < i.$$

The requirement $1 \le t_1, ..., t_m < i$ captures the idea that the input program is a DAG, because each parameter of an IR instruction is the index of some previous instruction. This also means that the first instruction must be nullary.

In addition to the sequence of instructions, a subset of program indices $R \subseteq \{1, ..., N\}$ are designated as **results**. An index $i \in R$ being present in the set means that the result of the instruction P_i must be materialized in the machine program, i.e. it cannot be coalesced into an intermediate computation of a machine instruction.

For each instruction P_i in a given IR program, we can build an **expression tree** (denoted $\text{tree}(P_i)$) by recursively replacing the indices t in the parameter list with the nodes they refer to:

$$\text{tree}(P_i) := \alpha\Big(\text{tree}\Big(P_{t_1}\Big), ..., \text{tree}\Big(P_{t_m}\Big)\Big)$$
$$\textbf{where } P_i = \alpha(t_1, ..., t_m)$$

(Expression trees cannot actually be materialized in an implementation, since they may be exponentially large without sharing, but they are useful concept in the specification of the problem.)

2) The machine program

We have a finite set C' of **machine opcodes** $\beta \in C'$ with associated arities. A **machine program** M has a similar structure to the input program: it consists of a sequence $\{M_1, ..., M_K\}$ of **machine instructions**, each of which consists of an opcode and a number of previous argument references that matches the arity.

$$M := \{M_1, ..., M_K\}$$
$$M_j := \alpha(r_1, ..., r_m), \qquad \beta \in C', \quad m = \text{arity}(\beta), \quad 1 \le r_1, ..., r_m < j.$$

As a convention, we use variables i and t as indexes of IR instructions and j and r for machine instructions.

A machine instruction is thought of as standing in for one or more IR instructions. For example, we could imagine a machine opcode FMA of arity 2, where $\text{FMA}(a, b, c)$ represents the computation $\text{add}(a, \text{mul}(b, c))$. We formalize this with the idea of a **machine definition** D, which associates each n-ary machine opcode β with a definition in terms of a tree of IR instructions with n free variables, written τ.

$$\begin{aligned} \tau &:= x_k && \text{(free variable)} \\ &\mid \alpha(\tau_1, ..., \tau_m) && \alpha \in C, \ m = \text{arity}(\alpha) \end{aligned}$$

$$\text{fv}(\tau) := \begin{cases} \{x_k\} & \text{if } \tau = x_k \\ \bigcup_{k=1}^{m} \text{fv}(\tau_k) & \text{if } \tau = \alpha(\tau_1, ..., \tau_m) \end{cases} \qquad \text{(set of free variables of a tree } \tau)$$

$$D_\beta := ((x_1, ..., x_m), \tau) \qquad \beta \in C', \ m = \text{arity}(\beta), \ \text{fv}(\tau) = \{x_1, ..., x_m\}$$

Given a machine definition $D_\beta = ((x_1, ..., x_m), \tau)$, we can apply it to a concrete m-tuple of trees, written as $D_\beta(\tau_1, ..., \tau_m)$, by substituting each x_k for τ_k in the body of the definition τ.

Given a machine program and a collection of machine definitions, we can define an analogous notion of **expression tree** for each instruction M_j.

$$\text{tree}(D, M_j) := D_\beta\Big(\text{tree}\Big(D, M_{r_1}\Big), ..., \text{tree}\Big(D, M_{r_m}\Big)\Big)$$
$$\textbf{where } M_j = \beta(r_1, ..., r_m)$$

For a fixed C, C', D, we say that an instruction M_j in a machine program **models** an IR instruction P_i if $\text{tree}(P_i) = \text{tree}(D, M_j)$. In other words, M_j computes the same expression tree as P_i.

Then we can say that a machine program M **models** an IR program (P, R) if for each designated result $i \in R$ there exists some instruction M_j that models P_i. In other words, every IR instruction designated as a result is computed by some machine instruction. Also note that the modeling requirement does not forbid the machine program from containing unnecessary instructions.

Notice that if an index i is not present in R then there is no requirement that the machine program materialize the value of P_i. This is what allows the compiler to use instructions like $\text{FMA}(a, b, c)$ when we don't need the intermediate computation $\text{mul}(b, c)$ to ultimately be saved in a register.

References

[1] G. S. H. Blindell, "Survey on Instruction Selection: An Extensive and Modern Literature Review." [Online]. Available: https://arxiv.org/abs/1306.4898